# DOCUMENTATION OF CORRECTIVE, PREVENTIVE, AND IMPROVEMENT ACTIONS FOR THE SOFTWARE

**PROJECT: Traduce Señas**

**TEAM MEMBERS:**
**Luis Alejandro Duarte Aldana**
**Juan Mauricio Suaza Solorzano**
**Juan Camilo Fierro Esquivel**

**NATIONAL LEARNING SERVICE – SENA**
**Software Analysis and Development**

2026

# Table of Contents

Introduction ..... 3
Definitions ..... 3
Methodology ..... 3
Corrective actions ..... 4
Preventive actions ..... 5
Improvement actions ..... 6
Lessons learned ..... 7
Follow-up ..... 7
Conclusions ..... 7
References ..... 8

# Introduction

This document records the actions that the Traduce Señas team has taken and plans to take to correct the defects found, prevent future problems, and continuously improve the software. It corresponds to the corrective, preventive, and perfective maintenance activities described in ISO/IEC 14764 (ISO, 2006).

The actions come from four sources: the code and security audit carried out during the migration to microservices, the test results, the project risk register, and the sprint retrospectives.

## Definitions

**Table 1**
*Types of action*

| Type | Definition | When it applies |
|---|---|---|
| Corrective | Removes the cause of a defect or failure that has already occurred | After an error is detected in testing, auditing, or use |
| Preventive | Removes the cause of a potential problem before it occurs | When an identified risk or a detected weakness exists |
| Improvement | Increases the quality, performance, or value of the software when no defect exists | As a result of retrospectives, metrics, or new needs |

# Methodology

Each finding is managed with the plan-do-check-act cycle:

1. **Identify:** record the finding with its source, date, and severity.
2. **Analyze the root cause:** ask “why?” until reaching the cause that, if removed, prevents it from recurring.
3. **Define the action:** owner, target date, and verifiable closure criterion.
4. **Execute** in a dedicated branch, with an automated test that reproduces the problem whenever possible.
5. **Verify** that the test passes and that no regressions appear (full suite green).
6. **Close** and record the lesson learned.

**Table 2**
*Severity scale*

| Severity | Criterion | Target response time |
|---|---|---|
| Critical | Exposes data or allows unauthorized access; the system does not work | Immediate, before any deployment |
| High | A main function fails or there is a relevant security risk | In the current sprint |
| Medium | Partial failure with an available workaround | In the next sprint |
| Low | Cosmetic or maintainability detail | According to backlog capacity |

## Corrective actions

The code and security audit of phase 4 of the migration to microservices found 2 critical issues, 2 high-severity issues, and 7 defects. The following table presents the actions taken.

**Table 3**
*Corrective actions executed*

| ID | Finding | Root cause | Action | Severity | Status |
|---|---|---|---|---|---|
| AC-01 | The service started even if JWT_SECRET was missing or weak | Permissive default value in the configuration | JWT_SECRET required (32+ characters), HS256 with issuer and audience | Critical | Closed |
| AC-02 | Databases and broker exposed with default passwords (guest/guest, postgres/postgres) | docker-compose intended for development only | Ports only on 127.0.0.1; passwords required from .env | Critical | Closed |
| AC-03 | Predictable recovery code with no attempt limit | Use of a non-cryptographic generator | crypto.randomInt, attempt counter, and strict limit on /api/auth | High | Closed |
| AC-04 | Any user could import or delete sign templates | Route without role verification | Only ADMIN (models.manage permission) can write templates | High | Closed |
| AC-05 | Open CORS and gateway routes with no service to handle them | Configuration inherited from the backend before the split into microservices | CORS_ORIGINS allowlist; orphan routes removed | Medium | Closed |
| AC-06 | Events without contract: invalid payloads could break consumers | Each service defined its own types | Typed event contract and validators in @traduce/shared | Medium | Closed |

| ID | Finding | Root cause | Action | Severity | Status |
|---|---|---|---|---|---|
| AC-07 | Achievement increments were lost with simultaneous translations | Non-atomic read and write | Atomic update with INSERT … ON CONFLICT DO UPDATE | Medium | Closed |
| AC-08 | Username not escaped in the email HTML | Direct interpolation in the template | escapeHtml function in the email templates | High | Closed |
| AC-09 | GET /api/profile/me failed if the profile did not exist | The profile is created per event and could arrive late | Profile created on the fly if it does not exist | Medium | Under verification |
| AC-10 | The downloader for the LSC-54 set silently skipped videos | Byte-range read error | Correction of the range calculation and test with a real sample | High | Closed |
| AC-11 | The download stopped with HTTP 429 | 16 simultaneous connections exceeded the server limit | 4 connections, larger blocks, and waiting on code 429 | Medium | Closed |

*Note.* Verification: the automated backend suite finished with 494 tests passed and 0 failed on October 7, 2026 (milestone of the audit). Current status, November 4, 2026: 918 tests passed, 0 failed and 13 skipped because they require PostgreSQL on the local machine.

## Preventive actions

Preventive actions address project risks before they turn into failures.

**Table 4**
*Preventive actions*

| ID | Risk it prevents | Preventive action | Owner | Indicator |
|---|---|---|---|---|
| AP-01 | Insufficient LSC training data (R-001) | Use the public LSC-54 set and capture our own samples with consent | Technical lead | ≥ 30 examples per sign |
| AP-02 | Model accuracy degraded without anyone noticing (R-002) | Measure accuracy per sign before publishing templates; review unrecognized signs monthly | Technical lead | Accuracy ≥ 85%; unrecognized rate < 20% |
| AP-03 | Confusion between signs with the same handshape | Evaluate with signers who did not take part in the templates; discard the signs that collide | Team | Confusion matrix per sign |
| AP-04 | Leakage of personal and biometric data (R-006) | Mandatory consent per sample; video processed on the device; ADMIN-only access | Team | 0 samples without consent |
| AP-05 | Single server without backup (R-007) | Daily backups and quarterly restore test | Deployment owner | Restore < 4 hours |
| AP-06 | Regressions when integrating changes | Mandatory automated tests before each pull request; code review | All | Suite green on every integration |

| ID | Risk it prevents | Preventive action | Owner | Indicator |
|---|---|---|---|---|
| AP-07 | Secrets in the repository | .env in .gitignore; .env.example without real values; review on every pull request | All | 0 secrets in the history |
| AP-08 | Vulnerable dependencies | Monthly npm audit and controlled update | Technical lead | 0 critical vulnerabilities |
| AP-09 | Role concentration in a single person (R-004) | Documentation for each service (README, runbook) and a technical onboarding guide | Team | Any team member can deploy |

## Improvement actions

**Table 5**
*Improvement plan*

| ID | Improvement | Expected benefit | Priority | Deadline |
|---|---|---|---|---|
| AM-01 | Structured logs with Pino and a correlation identifier across services | Much faster incident diagnosis | High | Next sprint |
| AM-02 | /health/ready endpoint that checks databases and broker | Deployments that detect dependency failures | High | Next sprint |
| AM-03 | Latency and error metrics with Prometheus | Measure compliance with the performance NFR (RNF) | High | Two sprints |
| AM-04 | Persist the classified errors in error_logs | Failure trend analysis (RF10) | Medium | Two sprints |
| AM-05 | Recognition engine that considers hand location and movement | Distinguish signs such as “good morning,” “good afternoon,” and “good night” | High | Next version |
| AM-06 | Two-hand sign recognition | Cover most of the signs in the LSC-54 set | Medium | Next version |
| AM-07 | Load tests with k6 in the pipeline | Detect degradation before production | Medium | Two sprints |
| AM-08 | Continuous integration in GitHub Actions (build + tests) | No change is integrated with failing tests | High | Closed: ci.yml in the backend and frontend repositories |
| AM-09 | Complete missing prototypes (notifications and 2 other screens) | Full design of the 21 screens | Low | Two sprints |
| AM-10 | Pilot test with deaf users at an educational institution | Real-world validation of specific objective 4 | High | Next version |

## Lessons learned

**Table 6**
*Lessons learned*

| Situation | Lesson |
|---|---|
| The requirements catalog reported traceability that was not real (17/17 and 77/79) | Every coverage metric must be recalculated from the source, not copied |
| The “rep_N” copies in the LSC-54 set were shifted versions of the same recording | Before downloading bulk data, study its structure with a sample |
| Wrist normalization removes hand location | Measure how distinguishable the signs are before promising a vocabulary |
| A development agent stopped midway through phase 5 due to a usage limit | Split the work into small commits and keep the state documented (ESTADO_TRABAJO.md) |
| The project budgets had four different totals | A data point must have a single source; the others reference it |

## Follow-up

The status of the actions is reviewed in every sprint retrospective. An action is only closed when there is verifiable evidence: an automated test that passes, a referenced commit, or a measured indicator. Open actions are kept in the technical backlog of the documentation repository (15-project-control/technical-backlog.md).

## Conclusions

The corrective actions closed the two critical and two high-severity issues found in the audit, and the test suite supports this. The preventive actions focus on training data quality and the protection of personal data, which are the highest risks of the project. The improvement plan prioritizes observability and a recognition engine that takes hand movement into account.

## References

Congress of the Republic of Colombia. (2012, October 17). *Ley Estatutaria 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales*. Official Gazette No. 48,587.

International Organization for Standardization. (2006). *Software engineering — Software life cycle processes — Maintenance* (ISO/IEC 14764:2006). ISO.

International Organization for Standardization. (2011). *Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models* (ISO/IEC 25010:2011). ISO.

Mora-Zarate, J. E., Garzón-Castro, C. L., & Castellanos Rivillas, J. A. (2025). LSC-54: A landmark-based dataset for Colombian Sign Language. *Data in Brief*. https://doi.org/10.1016/j.dib.2025.112145

OWASP Foundation. (2021). *OWASP Top 10: 2021*. https://owasp.org/Top10/

Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.